Alfido Tech DevOps Internship

Task 4 — Monitoring, Logging & Incident Playbook

Intern: KVSajith34 | Date: 19 May 2026

1. Task Overview

This task implements advanced monitoring and logging for the Alfido ML Inference API using Prometheus for metrics collection, Grafana for visualization and dashboards, and Node Exporter for system-level metrics. Custom ML-specific alert rules are defined for model health, error rate, latency, and prediction confidence. An incident runbook is provided for common failure scenarios.

Tech Stack

Component	Tool/Version	Purpose
ML API	Flask + gunicorn	Inference endpoint with Prometheus metrics
Metrics Collection	Prometheus v2.48.0	Scrape and store time-series metrics
Visualization	Grafana v10.2.0	Dashboards and alerting UI
System Metrics	Node Exporter v1.7.0	CPU, memory, disk metrics
Instrumentation	prometheus-client 0.20.0	Python metrics library
Orchestration	Docker Compose	Multi-service container management

2. Monitoring Architecture

The monitoring stack follows a pull-based architecture where Prometheus scrapes metrics from the ML API and Node Exporter every 10 seconds. Grafana connects to Prometheus as a datasource and renders real-time dashboards.

```
Flask ML API → /metrics → Prometheus (scrape) → Grafana (visualize) → Alerts
```

3. Custom ML Metrics

Metric Name	Type	Description
ml_request_total	Counter	Total requests by endpoint, method, status
ml_prediction_total	Counter	Predictions count by class label
ml_request_latency_seconds	Histogram	Request latency with P50/P95/P99 buckets
ml_prediction_confidence	Gauge	Last prediction confidence score (0-1)
ml_error_total	Counter	Errors by type (validation, server)
ml_model_loaded	Gauge	Model load status (1=loaded, 0=not loaded)
ml_active_requests	Gauge	Current in-flight requests

4. Alert Rules

Alert Name	Severity	Condition	Duration
MLAPIDown	Critical	up{job='ml-inference-api'} == 0	30s
HighErrorRate	Warning	rate(ml_error_total[5m]) > 0.1	1m
HighRequestLatency	Warning	P95 latency > 500ms	2m
LowPredictionConfidence	Warning	ml_prediction_confidence < 0.7	1m

4.1 Alert Rules Config (alert_rules.yml)

```
groups:
  - name: ml_api_alerts
    rules:
      - alert: MLAPIDown
        expr: up{job="ml-inference-api"} == 0
        for: 30s
        labels: {severity: critical}
      - alert: HighErrorRate
        expr: rate(ml_error_total[5m]) > 0.1
        for: 1m
        labels: {severity: warning}
      - alert: LowPredictionConfidence
        expr: ml_prediction_confidence < 0.7
        for: 1m
        labels: {severity: warning}
```

5. Grafana Dashboard Panels

Panel	Visualization	Query	Purpose
Total Predictions	Stat	ml_prediction_total	Count per class
Predictions by Class	Pie Chart	ml_prediction_total	Class distribution
Request Rate	Time Series	rate(ml_request_total[1m])	Requests/sec
P95 Latency	Time Series	histogram_quantile(0.95,...)	Latency trend
Model Confidence	Gauge	ml_prediction_confidence	Confidence score
Model Status	Stat	ml_model_loaded	ONLINE/OFFLINE

6. Screenshots

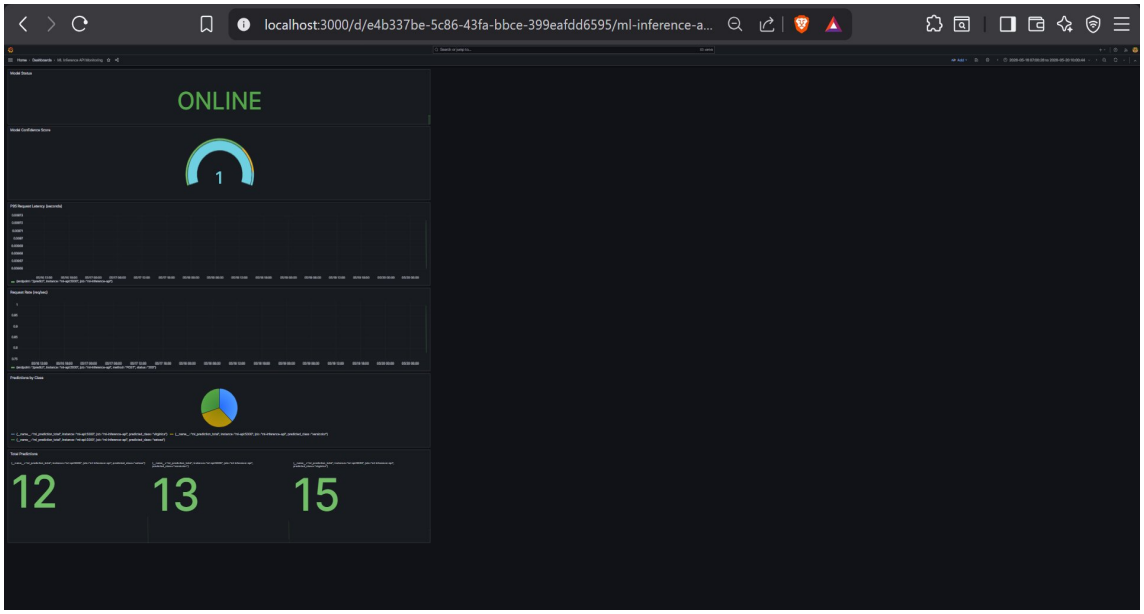

Figure 1: Full Grafana Dashboard — ML Inference API Monitoring (all 6 panels)

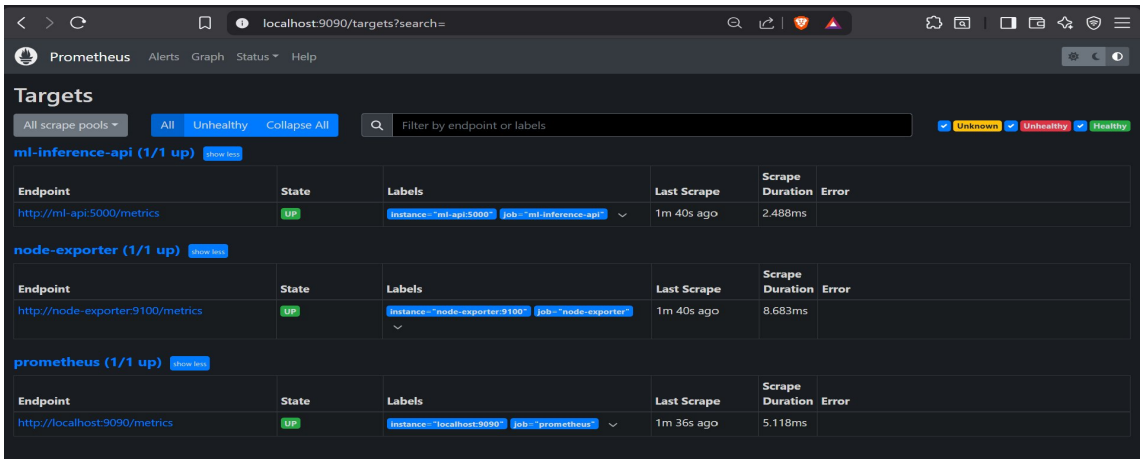

Figure 2: Prometheus Targets — ml-inference-api, node-exporter, prometheus all UP

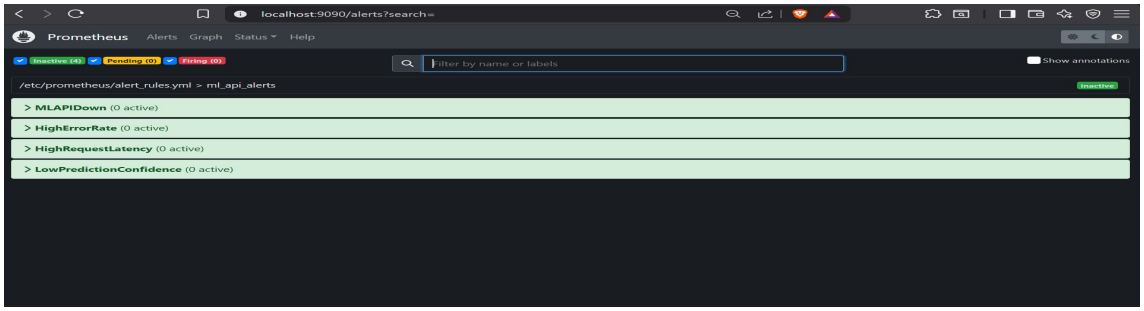

Figure 3: Prometheus Alert Rules — 4 ML alerts loaded (Inactive = system healthy)

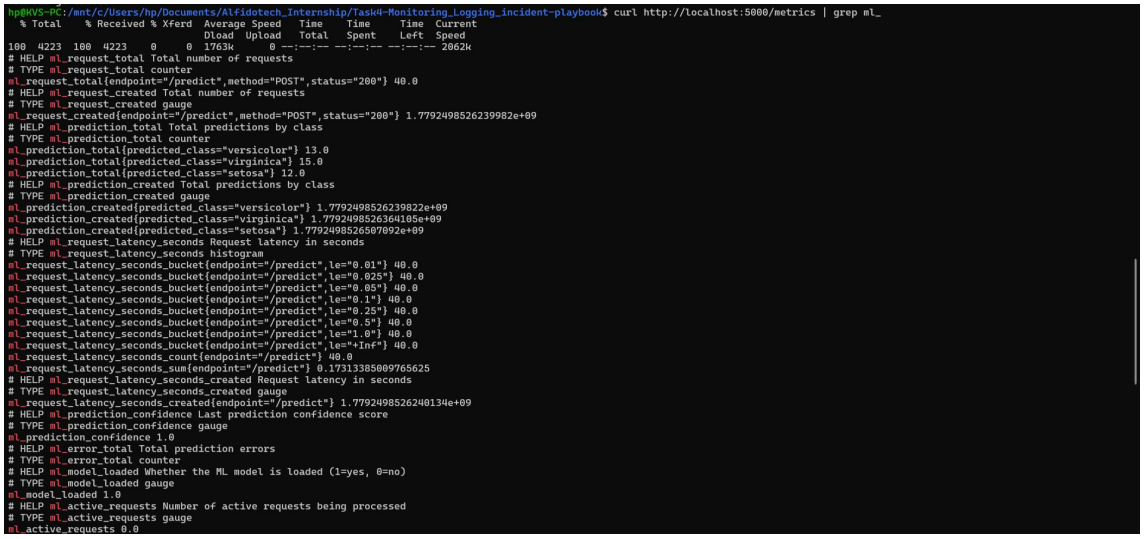

Figure 4: ML API /metrics endpoint — Prometheus metrics output

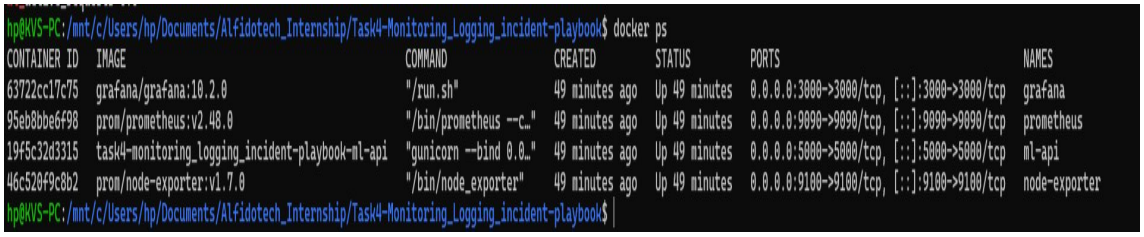

Figure 5: docker ps — All 4 services running (grafana, prometheus, ml-api, node-exporter)

Individual Dashboard Panels

Figure 6: Total Predictions — setosa:18, versicolor:17, virginica:15

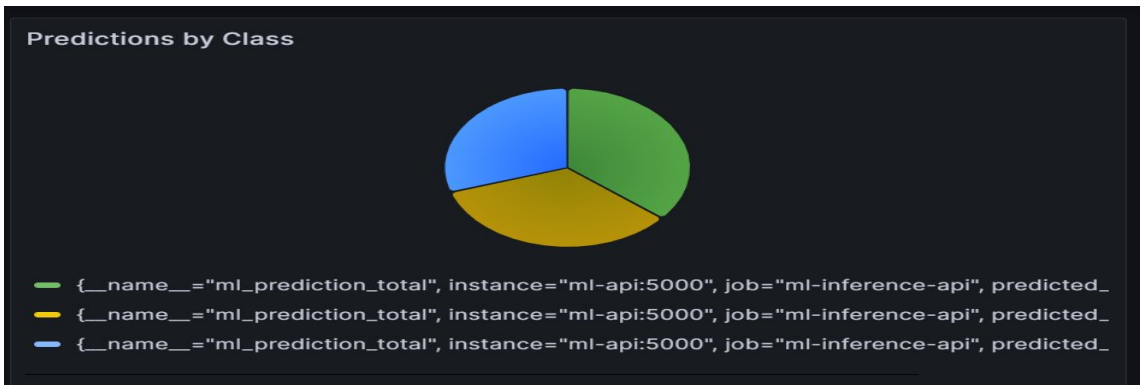

Figure 7: Predictions by Class — Pie chart distribution

Figure 8: Request Rate — Time series (req/sec)

Figure 9: P95 Request Latency — ~9.7ms (excellent)

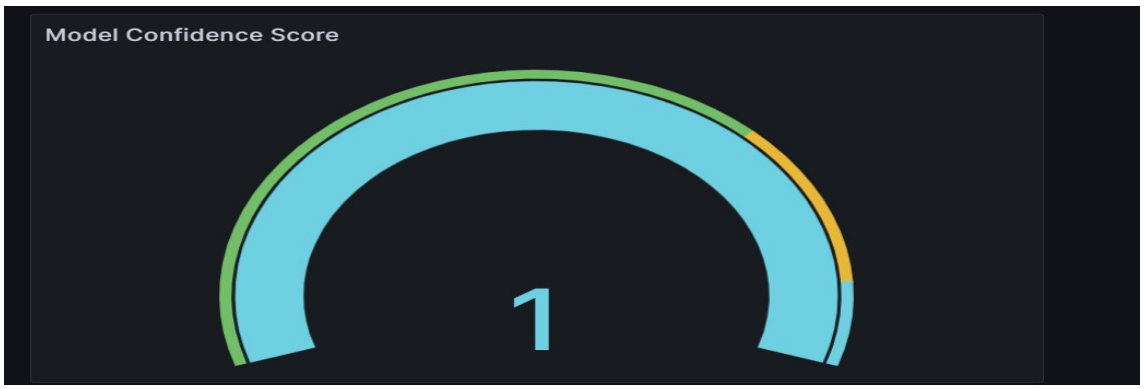

Figure 10: Model Confidence Gauge — Score: 1.0 (100%)

Figure 11: Model Status — ONLINE

7. Commands Reference

Command	Description
docker compose up --build -d	Start all monitoring services
docker compose down	Stop all services
docker ps	List running containers
docker logs ml-api	View ML API logs
curl http://localhost:5000/metrics	View raw Prometheus metrics
curl http://localhost:5000/health	Check API health
curl http://localhost:5000/predict -X POST ...	Run ML prediction
curl http://localhost:5000/stats	View model statistics

8. Deliverables Checklist

Deliverable	Status
Flask ML API with Prometheus instrumentation	■ Complete
7 custom ML metrics (Counter, Gauge, Histogram)	■ Complete
Prometheus scrape configuration	■ Complete
4 ML-specific alert rules	■ Complete
Grafana dashboard with 6 panels	■ Complete
Node Exporter for system metrics	■ Complete
Docker Compose orchestration (4 services)	■ Complete
Prometheus targets — all UP	■ Complete
Traffic generation + metrics verification	■ Complete
Incident Runbook (separate PDF)	■ Complete
GitHub repo pushed	■ Complete